

Exception Handling in Python

Introduction

When writing programs, errors can occur due to invalid input, missing files, incorrect operations, or unexpected situations. If these errors are not handled properly, the program may terminate abruptly.

Exception Handling is a mechanism in Python that allows us to handle runtime errors gracefully and continue the execution of the program without crashing.

It helps create robust, user-friendly, and professional applications.

What is an Exception?

An **Exception** is an error that occurs during the execution of a program.

Example

```
num1 = 10
```

```
num2 = 0
```

```
result = num1 / num2
```

```
print(result)
```

Output:

ZeroDivisionError: division by zero

The program stops immediately because of the error.

Why Exception Handling is Important

Without exception handling:

- ✗ Program crashes
- ✗ Poor user experience
- ✗ Data loss may occur
- ✗ Application becomes unreliable

With exception handling:

- ✓ Program continues running
- ✓ Better user experience

- ✓ Errors are managed gracefully
 - ✓ Easier debugging and maintenance
-

Types of Errors in Python

1. Syntax Errors

Errors caused by incorrect Python syntax.

```
if True
    print("Hello")
```

Output:

SyntaxError

2. Runtime Errors (Exceptions)

Errors that occur while the program is running.

```
print(10 / 0)
```

Output:

ZeroDivisionError

3. Logical Errors

The program runs successfully but produces incorrect results.

```
num = 10
```

```
if num > 20:
    print("Number is greater")
```

No output appears because the logic is incorrect.

Basic Exception Handling

Syntax

```
try:
    # Risky code

except:
    # Error handling code
```

Example

```
try:
    num1 = 10
    num2 = 0

    result = num1 / num2

    print(result)

except:
    print("An error occurred.")
```

Output:

An error occurred.

How try-except Works

```
Start
↓
Execute try block
↓
Error Occurred?
↓
Yes → Execute except block
↓
No → Continue normally
↓
End
```

Handling Specific Exceptions

It is recommended to handle specific exceptions rather than using a generic except block.

ZeroDivisionError

```
try:
    num = 10 / 0

except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Output:

Cannot divide by zero.

ValueError

Occurs when the user enters an invalid value.

```
try:
    age = int(input("Enter age: "))

except ValueError:
    print("Please enter a valid number.")
```

NameError

```
try:
    print(student_name)

except NameError:
    print("Variable not defined.")
```

TypeError

```
try:
    result = "10" + 5

except TypeError:
    print("Cannot combine string and integer.")
```

FileNotFoundError

```
try:
    file = open("data.txt", "r")

except FileNotFoundError:
    print("File does not exist.")
```

Multiple Exceptions

A single try block can handle multiple exceptions.

```
try:
```

```
num = int(input("Enter a number: "))
result = 100 / num
```

```
except ValueError:
    print("Invalid input.")
```

```
except ZeroDivisionError:
    print("Division by zero is not allowed.")
```

Exception as Object

Store exception details inside a variable.

```
try:
    num = 10 / 0
```

```
except Exception as e:
    print("Error:", e)
```

Output:

Error: division by zero

Using else Block

The else block executes only if no exception occurs.

Syntax

```
try:
    pass
```

```
except:
    pass
```

```
else:
    pass
```

Example

```
try:

    num = int(input("Enter a number: "))
```

```
except ValueError:
```

```
print("Invalid Input")
```

else:

```
print("You entered:", num)
```

Using finally Block

The finally block executes whether an exception occurs or not.

Syntax

try:

```
pass
```

except:

```
pass
```

finally:

```
pass
```

Example

try:

```
file = open("data.txt", "r")
```

except FileNotFoundError:

```
print("File not found")
```

finally:

```
print("Program Finished")
```

Output:

File not found

Program Finished

Complete Exception Handling Structure

try:

```
# Risky code
```

except:

```
# Handle error

else:
    # Runs if no error

finally:
    # Always executes
```

Example Program

```
try:

    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))

    result = num1 / num2

except ValueError:

    print("Please enter valid numbers.")

except ZeroDivisionError:

    print("Division by zero is not allowed.")

else:

    print("Result:", result)

finally:

    print("Calculation completed.")
```

Raising Exceptions

Python allows developers to create exceptions manually.

Syntax

```
raise ExceptionType("Message")
```

Example

```
age = int(input("Enter age: "))
```

```
if age < 18:  
    raise ValueError("Age must be 18 or above.")
```

Output:

ValueError: Age must be 18 or above.

Custom Exception Example

```
balance = 1000
```

```
withdraw = int(input("Enter amount: "))
```

```
if withdraw > balance:  
    raise ValueError("Insufficient Balance")
```

Common Built-in Exceptions

Exception	Description
ZeroDivisionError	Division by zero
ValueError	Invalid value
TypeError	Invalid data type operation
NameError	Undefined variable
IndexError	Invalid list index
KeyError	Dictionary key not found
FileNotFoundError	File does not exist
AttributeError	Invalid object attribute
ImportError	Module import failure

Practical Example 1: Safe Calculator

try:

```
num1 = float(input("Enter First Number: "))  
num2 = float(input("Enter Second Number: "))  
  
print("Result:", num1 / num2)
```


except ValueError:

```
    print("Enter valid numbers.")
```

except ZeroDivisionError:

```
    print("Cannot divide by zero.")
```

Practical Example 2: File Reader

try:

```
    with open("student.txt", "r") as file:  
        print(file.read())
```

except FileNotFoundError:

```
    print("File not found.")
```

Practical Example 3: Login System

```
correct_password = "python123"
```

try:

```
    password = input("Enter Password: ")  
  
    if password != correct_password:  
        raise ValueError("Invalid Password")  
  
    print("Login Successful")
```

except ValueError as e:

```
    print(e)
```

Mini Project: Student Marks Validator

try:

```
    marks = int(input("Enter Marks: "))
```

```
if marks < 0 or marks > 100:  
    raise ValueError("Marks must be between 0 and 100")
```

```
print("Marks Accepted")
```

```
except ValueError as e:
```

```
    print(e)
```

Mini Project: ATM Withdrawal System

```
balance = 5000
```

```
try:
```

```
    amount = int(input("Enter Withdrawal Amount: "))
```

```
    if amount > balance:  
        raise ValueError("Insufficient Balance")
```

```
    balance -= amount
```

```
    print("Withdrawal Successful")  
    print("Remaining Balance:", balance)
```

```
except ValueError as e:
```

```
    print(e)
```

Best Practices

Catch Specific Exceptions

 Good

```
except ValueError:
```

 Avoid

```
except:
```

Use Meaningful Error Messages

```
except ZeroDivisionError:  
    print("Division by zero is not allowed.")
```

Use finally for Cleanup

```
finally:  
    file.close()
```

Avoid Hiding Errors

Bad Practice:

```
except:  
    pass
```

Real-Life Applications

Banking Systems

Handle insufficient balance errors.

Student Management Systems

Validate marks and attendance.

File Management Applications

Handle missing files.

Login Systems

Validate user credentials.

E-Commerce Websites

Handle payment and transaction errors.

Learning Outcomes

After completing this topic, students will be able to:

- ✓ Understand runtime errors and exceptions
- ✓ Use try and except blocks
- ✓ Handle multiple exceptions
- ✓ Use else and finally blocks

- ✓ Raise custom exceptions
 - ✓ Create robust applications
 - ✓ Prevent program crashes
 - ✓ Build professional Python projects
-

Summary

Exception Handling is one of the most important concepts in Python programming. It allows developers to detect, manage, and recover from runtime errors without terminating the program unexpectedly.

The main keywords used in exception handling are:

- try
- except
- else
- finally
- raise

By mastering exception handling, developers can build secure, reliable, and user-friendly applications that can gracefully handle unexpected situations and errors.